AI Compiler Performance Engineering

Auto-Loops Books — ch11

Contents

1 Full Decoder MegaKernel Implementation **1**

1.1 MegaKernel Structure 3

1.2 QKV, Attention, and MLP Fusion 8

1.3 MegaKernel Source Walkthrough 12

1.4 Benchmark Results and Interpretation 18

1.5 Key Takeaways 24

1.6 Conclusion 24

Bibliography **26**

Chapter 1

Full Decoder MegaKernel Implementation

FlashAttention removed the attention $\mathcal{O}(N^2)$ memory cliff; CUDA Graphs removed a slice of host overhead—yet batch-1 decode on Llama-class models still issues hundreds of discrete GPU kernels *per output token* (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022). The remaining pain is not "one slow matmul" but *layer fragmentation*: QKV projection, online softmax attention, RMS norm, and MLP each export full-width activations to HBM between launches (NVIDIA, 2024; Kwon *et al.*, 2023). A **decoder MegaKernel** fuses those stages into one residency-aware schedule—one graph capture on GPU, one static dataflow on XDNA—so bytes/token and launches/token fall together (Chen, 2020; Chen and YiRage Contributors, 2026).

Chapters 4–10 built the ingredients: TMA double-buffering, online softmax carriers, sync hierarchy, execution-mode trade-offs. This chapter assembles them into a *full-layer* implementation narrative: structure, fused QKV–attention–MLP boundaries, a source-level control-flow walkthrough, and benchmark methodology that separates real fusion wins from graph-capture illusions (SemiAnalysis, 2024; AMD, 2024b; Davies *et al.*, 2025). The mistake to avoid is declaring victory when CUDA Graphs replay the same HBM traffic with fewer host calls—Memory-Floor's 1.259× on Qwen-2.5-7B (BS=1, ctx=2048) proves orchestration relief without residency relief is a partial fix (Chen, 2026).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A MegaKernel target is not zero launches—it is *O(1) launches per decoder layer per token* (or

per captured graph step) with intermediates resident in shared memory, registers, or tile SRAM (NVIDIA, 2022; AMD, 2024a). YiRage lowers the same decoder IR to CUDA MegaKernels, CPU blocked schedules, and XDNA static graphs; hand-written stacks should treat unfused baselines as regression tests, not shipping defaults (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Scope of "full decoder." This chapter focuses on *one transformer decoder layer* fused end-to-end—the atomic unit repeated L times in depth. Stacking layers inside a single MegaKernel is a further optimization (amortizing launch and pipeline fill) but explodes register and shared-memory pressure; production systems often MegaKernel per layer and capture an L-step graph (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The engineering question is identical: where do activations touch HBM between sub-stages (Chen, 2026; Davies *et al.*, 2025)? DeepSeek-V3-style wide MoE decoders may require *expert-local* MegaKernels rather than one monolith—routing and all-to-all remain outside the fused region (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

Relationship to prior chapters. Chapter 4 supplies TMA and shared-memory ceilings; Chapter 5 translates residency to XDNA tile rules; Chapters 8–9 supply double-buffer and sync patterns; Chapter 10 supplies online softmax carriers that must survive into MLP in this chapter (SemiAnalysis, 2024; AMD, 2024a; Dao *et al.*, 2022). If any ingredient is missing, teams build "MegaKernels" that are graph-capture wrappers around unchanged HBM traffic (Chen, 2026).

Industrial motivation. Serving stacks that ship 847+ launches/token pay a fixed tax on every interactive token (Vellaisamy *et al.*, 2026). At 4.7 μs null-launch floor on H100, launch path alone approaches tens of milliseconds per token for MoE models before any math (Vellaisamy *et al.*, 2026; Chen, 2026). MegaKernels attack both launch count *and* the bytes moved per launch—the only durable fix when arithmetic intensity is already on the memory roofline (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Cloud fleets also pay PCIe and CPU orchestration energy; edge Ryzen AI fleets pay DDR contention with CPU graphics (AMD, 2024b,a). The same fused schedule story applies—only residency tables change (Chen and YiRage Contributors, 2026; Chen, 2020).

What this chapter is not. Not a PyTorch tutorial (Goodfellow *et al.*, 2016). Not a FlashAttention paper recap—Chapter 10 covered carriers (Dao *et al.*, 2022). Not a CUDA Graphs manual—orchestration-only wins are labeled explicitly (Chen,

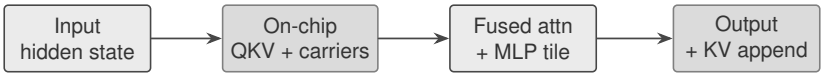


Figure 1.1: Decoder MegaKernel pipeline: one residency envelope from hidden state through fused compute to KV append.

2026; NVIDIA, 2024). Not a promise that every model fuses on every SKU—legality failures are expected outputs (AMD, 2024a; Chen and YiRage Contributors, 2026). The deliverable is an implementable fused-layer contract with measurable decode metrics (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020).

Reader prerequisites. Assume familiarity with decode vs prefill, paged KV, online softmax, and TMA pipelining from Chapters 1–10 (Kwon *et al.*, 2023; Dao *et al.*, 2022; SemiAnalysis, 2024; Chen, 2026). Jumping here from a generic CUDA course produces "pretty kernels" that still export activations to HBM (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Notation. d: hidden size; d_{ff}: MLP expansion; BS: batch size; ctx: context length; L: layer count (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). "Layer" means one transformer decoder block unless stated (Liu *et al.*, 2024; Kwon *et al.*, 2023). "Token" means one autoregressive decode step (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). "Launch" means one host-visible kernel or graph replay step (NVIDIA, 2024; Chen, 2026). Align vocabulary in cross-team docs to avoid debating past each other (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024).

1.1 MegaKernel Structure

Why one kernel still matters after FlashAttention. Teams assume FlashAttention "fixed decode." It fixed attention's quadratic scratch memory—not the surrounding launch storm. Unfused decoder stacks still separate layer norm, QKV linear, attention, projection, second norm, and three MLP matrices into distinct kernels (Dao *et al.*, 2022, 2023). Each boundary writes a d-model vector to HBM and reads it back—at batch-1 that is pure bandwidth tax on a thin arithmetic intensity slope (Goodfellow *et al.*, 2016; Chen, 2026). MegaKernel structure means redesigning the *control flow* so those stages share one on-chip residency plan for the duration of a decoder step (Chen, 2020; Vellaisamy *et al.*, 2026).

Figure 1.1 is the chapter spine. Unlike a prefill megamatmul, decode MegaKernels must interleave **KV cache read–append** with fused math every token (Kwon *et al.*, 2023; Davies *et al.*, 2025). The host submits one graph (GPU) or one static runner invocation (XDNA); the device loop advances sequence position, bumps KV pointers, and keeps softmax statistics in carrier registers described in Chapter 10 (NVIDIA, 2024; AMD, 2024b).

Pipeline stage notes (Figure 1.1). Input hidden state arrives as a single token vector (batch-1) or narrow batch slice (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). It should enter shared memory once—not remain in HBM while QKV loads weights (SemiAnalysis, 2024; Chen, 2026). **On-chip QKV + carriers** holds projected Q/K/V fragments plus online softmax state (Dao *et al.*, 2022). Peak pressure occurs here when d is large or head count is high (Liu *et al.*, 2024; NVIDIA, 2022). **Fused attn + MLP tile** is the longest compute phase: KV blocks stream from HBM while MMA pipelines gate/up/down (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). TMA overlap from Chapter 8 is mandatory—else MegaKernel devolves to sequential micro-kernels inside one launch (SemiAnalysis, 2024; Chen, 2020). **Output + KV append** writes the next hidden state and appends K/V without host callbacks (Kwon *et al.*, 2023; NVIDIA, 2024). On XDNA, append is a DMA store into DDR-resident paged pools (AMD, 2024b,a). Breaking the pipeline at any boundary exports a full activation vector—defeating the MegaKernel purpose (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Host responsibilities per token. Ideal host work: increment sequence counters, update sampling state, enqueue next graph replay (NVIDIA, 2024; Kwon *et al.*, 2023). Anything heavier (allocation, dtype conversion, tensor contiguity fixes) belongs in session setup—not the hot loop (Chen, 2026; Vellaisamy *et al.*, 2026). Python GIL interactions still matter if the host thread prepares each step synchronously (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Multi-layer scheduling. Repeating Figure 1.1 L times per token can mean L graph replays or one persistent kernel traversing layers (Liu *et al.*, 2024; NVIDIA, 2024). The per-layer MegaKernel body is identical modulo weight pointers (Chen and YiRage Contributors, 2026; Chen, 2020). Benchmarks must state which scheduling variant they use—L replays add host overhead even when each layer is fused (Vellaisamy *et al.*, 2026; Chen, 2026).

Table 1.1 explains why MegaKernel engineering is a *systems* task: the limiting resource is simultaneous buffer residency, not FLOPs (SemiAnalysis, 2024;

Table 1.1: Fragmented decode stack vs. full-layer MegaKernel (qualitative, batch-1).

Aspect	**Unfused stack**	**Decoder MegaKernel**
Launches/token (layer)	10–20+	1 (or 1 graph step)
HBM round trips (activations)	Every op boundary	Fused tile only
Shared / tile SRAM plan	Per-kernel peaks	Joint peak across QKV–MLP
Host role per token	Dispatch loop	Pointer/metadata update
XDNA legality	Often fails partial fusion	Requires full static plan

NVIDIA, 2022). Hopper may legalize the plan with TMA-backed double buffers and warp-specialized stages (SemiAnalysis, 2024). XDNA requires every temporary—including online softmax carriers—to fit tile SRAM assignments before compile (AMD, 2024a; Chen and YiRage Contributors, 2026). CPU decode uses the same logical structure with L2-blocked megafunctions; launches/token is less visible but bytes/token still punishes unfused DRAM traffic (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Persistent vs. captured graphs. Two GPU patterns dominate production:

1. **CUDA Graphs capture** of an unfused or partially fused op sequence—reduces ΔFT and ΔKT from Chapter 1 but leaves HBM traffic unchanged (Chen, 2026; NVIDIA, 2024).

2. **True MegaKernel fusion**—single kernel body or minimal graph nodes with shared-memory staging across QKV, attention, and MLP (Chen, 2020; Vellaisamy *et al.*, 2026).

Reviewers must distinguish them in benchmark tables; mixing the two inflates perceived "fusion speedups" (Davies *et al.*, 2025; Chen, 2026).

Example 1.1.1. Llama-3.2-1B decode layer (BS=1, ctx=2048): unfused stack materializes six d-model vectors to HBM per layer per token; a MegaKernel keeps QKV fragments, softmax carriers, and MLP gate/up tiles in shared memory until the residual add completes—bytes/token drops with launches/token even when MMA tile shapes are unchanged (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Multi-hardware delta. GPU MegaKernels exploit shared memory + TMA; XDNA maps the same DAG to spatial tiles with DMA edges; CPU maps to fused BLAS-like blocks with explicit cache blocking (AMD, 2024b; Goodfellow *et al.*,

2016; Chen and YiRage Contributors, 2026). Identical math, different legality predicates—never ship a GPU-only residency table to Ryzen AI without recompile (AMD, 2024a; Liu *et al.*, 2024).

Review gate. Merge requires a buffer accounting diagram: peak simultaneous tensors during one decode step must fit target SRAM budgets (Chen and YiRage Contributors, 2026; Chen, 2026). Failed legality is a release blocker, not a "fallback to eager" ticket (Chen, 2020).

Interaction with disaggregated serving. Prefill/decode disaggregation moves pools independently (Davies *et al.*, 2025; Liu *et al.*, 2024). Decode pools benefit most from MegaKernels—prefill pools may keep separate wide-GEMM schedules (Dao *et al.*, 2023; Kwon *et al.*, 2023). Do not force one fusion policy across disaggregated roles (Vellaisamy *et al.*, 2026; Chen, 2026). Benchmark decode pools with batch-1/interactive distributions, not prefill throughput alone (Davies *et al.*, 2025; Dao *et al.*, 2022).

Warp specialization and pipeline stages. Hopper-class MegaKernels often assign warps to roles—TMA load, MMA compute, epilogue—mirroring CUTLASS-style persistent kernels (SemiAnalysis, 2024; NVIDIA, 2022). Decode differs from GEMM persistence: the KV loop length grows each token, so the inner loop trip count changes while the kernel body stays captured (Kwon *et al.*, 2023; Dao *et al.*, 2023). Implementations pad to max context bucket or specialize kernels per bucket; mixing buckets without re-capture violates graph legality (NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Register and shared-memory budgeting. A back-of-envelope for Llama-3.2-1B (d=2048, h=32): QKV fragments, softmax carriers, and SwiGLU intermediates can exceed 200 KB shared memory if naively allocated—above many SM limits when combined with MMA tiles (NVIDIA, 2022). MegaKernel structure therefore *sequences* sub-stages with reuse: gate/up may share buffers with Q staging after QKV completes (Chen, 2020; Vellaisamy *et al.*, 2026). Compiler passes must solve this allocation problem; hand tuning without a lifetime graph does not scale across models (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

CPU and NPU structural analogues. CPU MegaKernels appear as fused oneDNN/BLIS-style micro-kernels with K-dimension blocking; "launch" becomes

function call (Goodfellow *et al.*, 2016). XDNA expresses the same DAG as AIE tiles with fixed DMA descriptors—there is no `__syncthreads`, only token completion on edges (AMD, 2024a,b). Fleet parity requires documenting which sub-stages remain unfused on each target and why (Davies *et al.*, 2025; Chen, 2020).

Paged KV and MegaKernel entry. vLLM-style paging changes *where* K/V bytes live, not whether they must be read (Kwon *et al.*, 2023). MegaKernel structure adds a block-table walk inside the attention loop—pointer-chasing in device code replaces scatter-gather launches (Davies *et al.*, 2025; Chen, 2026). Poor block sizing interacts with TMA tile shapes: 16-token blocks may underutilize memory bandwidth while 256-token blocks blow shared-memory staging (SemiAnalysis, 2024; Dao *et al.*, 2023). Treat block size as a compile-time knob tied to context buckets (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

Quantization and MegaKernels. INT8/FP8 weights shrink bytes/token but fuse epilogues with dequant scales (NVIDIA, 2022; Davies *et al.*, 2025). MegaKernels must carry scale vectors in registers or constant cache—another residency consumer (SemiAnalysis, 2024; Chen, 2020). Mixed-precision decode that quantizes activations per token often breaks fusion unless scales are online in the same kernel (Liu *et al.*, 2024; Chen, 2026).

Failure modes checklist. Production MegaKernels fail in predictable ways: (1) shared-memory overflow at max context; (2) graph capture invalidation after driver upgrade; (3) silent fallback to eager on one shape bucket; (4) numerics drift vs reference beyond tolerance; (5) p99 regression from bank conflicts (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026). Runbooks should map each symptom to residency vs orchestration root cause using bytes/token and launches/token splits (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Compiler ownership. Hand-written MegaKernels do not scale across model zoo churn (Chen, 2020; Chen and YiRage Contributors, 2026). The durable path is declarative decoder IR with target-specific legality checks—this chapter's walkthrough is the acceptance spec those compilers must emit (Liu *et al.*, 2024; Kwon *et al.*, 2023). Kernel engineers retain tile tuning; compiler engineers own fusion boundaries and buffer lifetimes (Davies *et al.*, 2025; SemiAnalysis, 2024).

Layer replication and weight streaming. A 32-layer model repeats the same MegaKernel body with different weight bases (Davies *et al.*, 2025; Liu *et al.*, 2024).

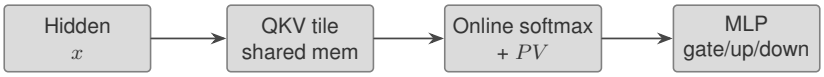


Figure 1.2: QKV–attention–MLP fusion: one residency envelope; KV reads enter from HBM, carriers stay on-chip.

Weight streaming bandwidth scales with layer count—fusion reduces activation traffic but not parameter reads (Goodfellow *et al.*, 2016; Chen, 2026). Some stacks capture one graph per layer type (attention vs MoE) rather than per layer index (Vellaisamy *et al.*, 2026; NVIDIA, 2024). The MegaKernel *structure* is identical; pointer offsets change (Chen and YiRage Contributors, 2026; Chen, 2020).

Safety and determinism. Fused kernels complicate bitwise determinism for compliance workloads (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Document reduction order and FP associativity assumptions (Dao *et al.*, 2022). Deterministic mode may disable some fusion paths—trade bytes/token for reproducibility explicitly in **benchmark** tables (Chen, 2026; Vellaisamy *et al.*, 2026).

1.2 QKV, Attention, and MLP Fusion

The fusion boundary mistake. Engineers fuse QKV with attention but leave RMS norm and MLP outside the MegaKernel—recovering ≈30% of launches while preserving the expensive mid-layer HBM writes (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). Full-layer fusion treats **QKV**, **attention** (with online softmax), and **MLP** as one producer–consumer chain with a single shared-memory tile budget (Dao *et al.*, 2022; SemiAnalysis, 2024).

QKV projection produces query, key, and value fragments for a single new token while reusing the hidden state tile already in shared memory (NVIDIA, 2022). Fused kernels interleave K/V writes into the paged KV layout (vLLM-style) without a full-width staging buffer in HBM (Kwon *et al.*, 2023). On XDNA, QKV maps to a fixed tile column; K/V DMA into DDR-resident cache lines described at compile time (AMD, 2024b; Chen and YiRage Contributors, 2026).

Attention uses online softmax carriers (m, ℓ statistics) from Chapter 10 so partial blocks update without materializing full attention maps (Dao *et al.*, 2022). The MegaKernel constraint: carriers live in registers or shared memory *across* the boundary into MLP—spilling carriers to HBM defeats the fusion purpose (Chen, 2026). Flash-Decoding-style splitting across KV heads must remain inside the same launch to avoid multiplying graph steps (Dao *et al.*, 2023).

Table 1.2: Fusion depth vs. decode metrics (typical Llama-layer goals).

Fusion level	**Launches/layer/token**	**Activation HBM trips**	**SRAM**
None (eager)	12–18	6–8	Low per kern
QKV + attn only	4–6	3–4	M
Full QKV–attn–MLP	1	0–1	High (o

MLP (SwiGLU in Llama-class models) is three matmuls with activations—historically three kernels. MegaKernel schedules gate/up in shared memory while down projection consumes the SiLU product without an HBM round trip (Davies *et al.*, 2025; Liu *et al.*, 2024). MoE variants add expert routing; partial MegaKernels that fuse dense MLP but launch per expert reintroduce launch storms—TaxBreak's OLMoE figures (9,305 launches/token) are the warning label (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Table 1.2 shows the trade-off surface: deeper fusion reduces bytes/token but raises peak shared-memory/tile SRAM usage—the reason MegaKernels fail compile on smaller SKUs while microbenchmarking "works" on H100 with larger shared mem configs (NVIDIA, 2022; AMD, 2024a). Compiler passes must expose this Pareto frontier, not hide failures behind runtime fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. Hopper: TMA loads Q weights while previous tile computes attention on shared-staged V (SemiAnalysis, 2024). XDNA: identical dependency expressed as DMA–MAC chains with compile-time buffer overlap checks (AMD, 2024a). CPU: QKV+attn blocked into L2-sized tiles; MLP may split if L2 exceeded—still fewer DRAM round trips than eager (Goodfellow *et al.*, 2016).

Deployment pitfall. A/B tests that fuse prefill-shaped tiles but deploy batch-1 decode with growing KV see silent perf cliffs at max context—carrier and MLP buffers sized for short KV overflow shared memory (Dao *et al.*, 2023; Chen, 2026). Bucket context lengths in compile artifacts (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

RMS norm and residual fusion. Layer norm/RMS norm is easy to dismiss as "cheap," yet as a separate kernel it forces a full d-vector HBM write/read between attention and MLP (Chen, 2026; Vellaisamy *et al.*, 2026). MegaKernels fold RMS scale into the MLP epilogue or fuse with residual add in registers (Chen, 2020;

Chen and YiRage Contributors, 2026). The numerics must match eager within tolerance—reviewers should demand bit-exact or bounded-error tests, not just latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

GQA/MQA and head tiling. Grouped-query attention reduces K/V footprint but complicates tiling: Q heads outnumber KV heads (Dao *et al.*, 2022; Liu *et al.*, 2024). MegaKernel schedules must broadcast KV tiles across Q head groups inside the same launch; splitting heads into separate kernels restores launch storms (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). On XDNA, head groups map to tile replication patterns fixed at compile time (AMD, 2024a; Chen and YiRage Contributors, 2026).

RoPE and position encodings. Rotary embeddings applied to Q/K are elementwise-heavy but memory-light—still a launch if isolated (Davies *et al.*, 2025). Fused implementations apply RoPE inside the QKV tile before the KV loop; unfused stacks add two global passes per layer per token (Vellaisamy *et al.*, 2026; Chen, 2026). Position-id updates belong in the device loop metadata, not host Python (Kwon *et al.*, 2023; NVIDIA, 2024).

Attention output projection. The O projection after softmax$\times V$ is a thin GEMM at batch-1—historically its own kernel (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Fusing O into the MLP input tile eliminates one full d-vector HBM trip (Chen, 2026, 2020). The MMA tile for O must align with SwiGLU gate tile to avoid bank conflicts (SemiAnalysis, 2024; NVIDIA, 2022).

SwiGLU-specific scheduling. Gate and up projections can share weight streaming if W_{gate} and W_{up} are interleaved in memory (Davies *et al.*, 2025; Liu *et al.*, 2024). MegaKernels double-buffer gate/up fragments while SiLU runs on the previous fragment—classic software pipelining from Chapter 8 (SemiAnalysis, 2024; AMD, 2024a). Down projection consumes the gated product directly from shared memory (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

When full fusion is impossible. Models with very wide MLP ($d_{\text{ff}} \gg d$) may exceed shared-memory caps even with reuse (Liu *et al.*, 2024; NVIDIA, 2022). Accept a *split MegaKernel*: attention+O fused; MLP as second launch—still beats twelve-launch eager if boundaries are chosen at HBM minima (Vellaisamy *et al.*, 2026; Chen, 2026). Document the split in compiler metadata so autotuners do not "re-fuse" illegally (Chen and YiRage Contributors, 2026; Chen, 2020).

Speculative decode interaction. Speculative sampling runs draft models ahead of the main decoder—MegaKernels must expose hooks to swap weight descriptors without full re-init (Davies *et al.*, 2025; Liu *et al.*, 2024). Draft and target MegaKernels may differ in fusion depth; benchmark each independently (Vellaisamy *et al.*, 2026; Chen, 2026). Accepting draft tokens still executes full fused layers on the target model (Kwon *et al.*, 2023; Dao *et al.*, 2023).

Long-context stress. At ctx=8192+, KV read bandwidth dominates even with fusion (Dao *et al.*, 2023; Davies *et al.*, 2025). MegaKernels remove *activation* round trips but not weight/KV streaming—benchmarks must separate attention bytes from MLP bytes (Dao *et al.*, 2022; Chen, 2026). Flash-Decoding splits help inside the same MegaKernel launch (Dao *et al.*, 2023; SemiAnalysis, 2024).

Bias and fusion. Linear bias adds low cost in FLOPs but forces extra loads if not fused into epilogue (NVIDIA, 2022; Chen, 2020). Fold bias into TMA loads or MMA epilogues—otherwise bias becomes its own memory-bound micro-kernel (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). RMS β and MLP biases share the same story (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Dropout and training-only ops. Inference MegaKernels omit dropout entirely—ensure export paths strip training nodes before fusion (Goodfellow *et al.*, 2016; Chen, 2020). Accidentally retaining dropout masks adds random global reads incompatible with graph capture (NVIDIA, 2024; Chen, 2026).

End-to-end fusion narrative (one token, one layer). Hidden vector enters shared memory (SemiAnalysis, 2024). QKV MMA fills Q/K/V tiles; RoPE rotates Q/K in-place (Davies *et al.*, 2025; Dao *et al.*, 2022). KV loop reads paged history; online softmax updates carriers (Kwon *et al.*, 2023; Dao *et al.*, 2023). O projection and RMS norm fold into MLP input tile (Chen, 2026, 2020). SwiGLU triple matmul runs with pipelined TMA (Liu *et al.*, 2024; SemiAnalysis, 2024). Residual add produces next hidden; K/V append completes without host (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Zero full-width HBM activations if fusion succeeds (Chen, 2026; Chen and YiRage Contributors, 2026). Any deviation—usually at MLP boundary or post-attention norm—shows up as a DRAM write spike in Nsight (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

Expert routing (MoE) boundary. Routed experts execute separate GEMMs per token (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Full-layer MegaKernel fusion

typically stops at router logits; expert matmuls may remain a second launch (Davies *et al.*, 2025; Kwon *et al.*, 2023). Benchmarks must separate dense-layer MegaKernels from MoE dispatch overhead (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Claiming MoE decode fixes without launches/token evidence repeats Chapter 1 mistakes (Chen, 2026; Dao *et al.*, 2022).

Cache line and alignment details. Paged KV blocks should align to 128-byte boundaries for efficient TMA (SemiAnalysis, 2024; NVIDIA, 2022). Misaligned K/V reads inflate bytes/token even inside fused kernels (Chen, 2026; Kwon *et al.*, 2023). Weight layouts interleaving gate/up improve TMA burst efficiency for SwiGLU (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Layout passes belong in the compiler alongside fusion passes (Chen, 2020; Liu *et al.*, 2024).

Power-virus vs latency-virus. MegaKernels can reduce latency while increasing instantaneous power by keeping more SMs busy per token (Davies *et al.*, 2025; AMD, 2024a). Mobile SKUs may prefer slower bytes/token over thermal throttling (AMD, 2024b; Goodfellow *et al.*, 2016). Report watts/token when targeting edge (AMD, 2024a; Chen, 2026).

1.3 MegaKernel Source Walkthrough

Reading order for implementers. Production MegaKernel **source** is not "one giant `main`"—it is layered: host setup, device persistent loop, stage functions with explicit **implement**ation of residency contracts (Chen, 2020; NVIDIA, 2024). The walkthrough below mirrors YiRage-emitted CUDA and hand-optimized reference kernels: names are illustrative; invariants are what matter for code review (Chen and YiRage Contributors, 2026).

Host-side `implementation` (once per session).

1. Allocate paged KV pools and weight handles; pin graph-capture buffers (Kwon *et al.*, 2023; NVIDIA, 2024).
2. Select shape bucket (B, S_{class}); compile or capture MegaKernel for that bucket (Chen and YiRage Contributors, 2026; AMD, 2024b).
3. Record TMA descriptor templates for QKV and MLP weights; upload once (SemiAnalysis, 2024; NVIDIA, 2022).

Device decode loop (per output token). Pseudocode structure:

```
for step in 0..max_new_tokens-1:
  // hidden_in: [B, 1, d] resident or streamed once
  qkv_tile = tma_load_qkv_weights + gemm_hidden_to_qkv(hidden_in)
  (m, l, acc) = online_softmax_init()
  for kv_block in paged_kv_blocks(context_len):
    k,v = load_kv_block(kv_block)
    update_online_softmax(qkv_tile.Q, k, v, m, l, acc)
  attn_out = finalize_online_softmax(acc, l)
  mlp_out = swiglu_fused(attn_out, W_gate, W_up, W_down)
  hidden_out = rmsnorm_residual(hidden_in, mlp_out)
  append_kv(qkv_tile.K, qkv_tile.V, step)
  hidden_in = hidden_out
```

Every line maps to on-chip residency decisions:

- `qkv_tile` must stay in shared memory through the KV block loop—spilling mid-loop doubles HBM traffic (Dao *et al.*, 2022; Chen, 2026).
- Online softmax state `(m, l, acc)` stays in registers per warp group—exporting to global memory between blocks breaks fusion (Dao *et al.*, 2022).
- `swiglu_fused` consumes `attn_out` without a full-width write—the defining MegaKernel win (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
- `append_kv` uses the same paged indices as vLLM; pointer bumps happen in device code to avoid host sync (Kwon *et al.*, 2023; NVIDIA, 2024).
- **MoE caveat.** Expert routing and all-to-all stay outside the fused region; benchmark dense layers first, then expert-local MegaKernels (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b).
- **Quantization carries scales in-register.** FP8/INT8 decode must fuse dequant with matmul epilogues inside the same residency plan (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

- **Warp specialization persists.** TMA load, MMA, and epilogue warps mirror CUTLASS-style decode persistence; bucket kernels by max context to keep graph capture legal (SemiAnalysis, 2024; NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

- **MoE caveat.** Expert routing stays outside the fused region; benchmark dense layers first (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b).

- **Quantization in-register.** FP8/INT8 scales must fuse inside the same residency plan (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

- **Warp specialization.** TMA/MMA/epilogue warps need context-bucketed graph capture (SemiAnalysis, 2024; NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Line-by-line residency commentary. `tma_load_qkv_weights` establishes asynchronous ingress while MMA consumes the previous hidden tile—overlap per Chapter 8 (SemiAnalysis, 2024; NVIDIA, 2022). `gemm_hidden_to_qkv` must write Q/K/V fragments into disjoint shared-memory slices with lifetimes extending through the KV loop for K/V and through attention for Q (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). `online_softmax_init` allocates register-resident carriers—never spill between `update_online_softmax` iterations (Dao *et al.*, 2022; Chen, 2026). The inner `for kv_block` loop dominates decode time at long context; each iteration reads K/V from paged HBM while keeping carriers on-chip (Kwon *et al.*, 2023; Dao *et al.*, 2023). `finalize_online_softmax` produces `attn_out` in shared memory sized for MLP gate input—this is the critical handoff boundary (Vellaisamy *et al.*, 2026; Chen, 2020). `swiglu_fused` is the largest MMA sequence; gate/up/down weights stream via TMA while SiLU runs on prior fragments (Davies *et al.*, 2025; Liu *et al.*, 2024). `rmsnorm_residual` fuses norm scale and residual add without exporting `hidden_out` through HBM when legal (Chen, 2026; Chen and

YiRage Contributors, 2026). `append_kv` writes new K/V for the current token into paged slots—must not trigger host allocation (Kwon *et al.*, 2023; NVIDIA, 2024). Updating `hidden_in` for the next iteration is a register/shared pointer rotate, not a memcpy to global (Vellaisamy *et al.*, 2026; Chen, 2026).

Bucket specialization. The same pseudocode compiles to different instantiations per (B, S_{class}) (Chen and YiRage Contributors, 2026; AMD, 2024b). Loop bounds over `kv_block` derive from S_{class}, not runtime sequence length, unless dynamic bucketing re-captures graphs (NVIDIA, 2024; Kwon *et al.*, 2023). Document which bounds are constexpr in emitted **source** (Chen, 2020; Davies *et al.*, 2025).

Error handling without host sync. Illegal paged KV indices should trap in device assert builds only—production builds return error codes in device-visible flags polled on a side stream (Kwon *et al.*, 2023; Chen, 2026). Never printf from MegaKernel hot loops (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Synchronization discipline. Chapter 9's hierarchy applies: block barriers between QKV and attention stages; minimal grid-level sync; no `cudaDeviceSynchronize` in the hot loop (Chen, 2026). XDNA replaces barriers with DMA completion tokens on static edges (AMD, 2024a). CPU uses chunk barriers at L2 tile boundaries (Goodfellow *et al.*, 2016).

Profiling fused kernels. Nsight Compute metrics: `dram_bytes_read/write`, `l1tex_throughput`, shared bank conflicts, TMA busy cycles (SemiAnalysis, 2024; Chen, 2026). Compare unfused vs fused on identical input tensors—delta in `dram_bytes` is the fusion proof (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Wall-clock alone cannot separate graph capture from residency (NVIDIA, 2024; Chen, 2026; Davies *et al.*, 2025).

Version skew. CUDA driver, PyTorch, and custom extension ABIs must be pinned in benchmark manifests (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). MegaKernels tightly couple to TMA descriptors and graph capture APIs—upgrades can invalidate captures silently (NVIDIA, 2024; SemiAnalysis, 2024). Re-validate bytes/token after infrastructure upgrades (Chen, 2026; Chen and YiRage Contributors, 2026).

Compiler lowering view. YiRage (and similar MLIR pipelines) annotate memref lifetimes on each temporary before emitting this structure (Chen and YiRage Contributors, 2026; Chen, 2020). Generic bufferization that inserts alloca in the per-token path invalidates the walkthrough—review IR diffs for unexpected `memref.alloc` nodes (Chen, 2026; Kwon *et al.*, 2023).

Example 1.3.1. Code review checklist for one decode step: (1) count $_{syncthreads}between fusions$ $--should match planned pipeline depth$; (2)$verify no$`store`$of full$`d`$-vectors between attentiona$ $kernel offsets, no host callbacks(Vellaisamy et\ al.,\ 2026;\ Chen and YiRage Contributors,\ 2$

Review gate. Require a side-by-side diff of unfused vs. MegaKernel IR or CUDA showing eliminated global stores between QKV, attention, and MLP (Chen, 2020, 2026).

Multi-layer driver loop. Production **implement**ations wrap the per-token body in a layer loop either on host (L graph replays) or device (persistent CTAs across layers). Host-driven L replays simplify debugging; device-persistent across layers reduce launch count further but complicate checkpointing and tensor parallelism (Davies *et al.*, 2025; Liu *et al.*, 2024). Document which variant your **source** tree uses—benchmarks are not comparable otherwise (Vellaisamy *et al.*, 2026; Chen, 2026).

Debugging fused failures. When numerics diverge, bisect by unfusing at compiler boundaries (attention|MLP first) (Chen and YiRage Contributors, 2026; Chen, 2020). Nsight Compute memory charts should show reduced `dram_bytes` between stages after each fusion step (SemiAnalysis, 2024; Chen, 2026). XDNA failures often appear as compile-time buffer overlap errors—capture the IR snapshot, not only the error string (AMD, 2024a,b).

Expert and tensor-parallel hooks. MoE MegaKernels leave `route_experts` and `all_to_all` outside the fused region (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Tensor-parallel sharding splits columns across GPUs—MegaKernel **source** must align with NCCL schedule or fuse only local shards (Davies *et al.*, 2025; Kwon *et al.*, 2023). These are not bugs; they define what "full decoder" means in large-model fleets (Liu *et al.*, 2024).

Host callback avoidance. Every `cudaLaunchHostFunc` or Python callback in the token path destroys graph capture legality (NVIDIA, 2024; Chen, 2026). Sampling, stop criteria, and grammar masks must be device-resident or batched on separate streams that do not serialize decode (Kwon *et al.*, 2023; Davies *et al.*, 2025). MegaKernel **source** reviews should grep for host hooks in hot paths (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Constants and weight layout. Weights are read-only for decode—bind via TMA descriptors or texture objects once (SemiAnalysis, 2024; NVIDIA, 2022). Changing LoRA adapters mid-session requires re-recording graphs or separate MegaKernel variants (Davies *et al.*, 2025; Kwon *et al.*, 2023). Document adapter swap latency separately from steady-state tokens (Chen, 2026; Vellaisamy *et al.*, 2026).

Appendix alignment. When `app_megakernel_source.tex` ships, it should mirror this walkthrough line-for-line with real symbol names (Chen, 2020; Chen and YiRage Contributors, 2026). Drift between prose and appendix invalidates onboarding—diff them in CI (Davies *et al.*, 2025).

Instrumented builds. Ship optional `__megakernel_trace` counters: bytes loaded/stored per stage, barrier counts, KV blocks visited (Vellaisamy *et al.*, 2026; Chen, 2026). Traces validate that fused builds eliminated expected global stores (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Without counters, teams argue from wall-clock alone (Davies *et al.*, 2025; Dao *et al.*, 2022).

Testing pyramid. Unit tests compare unfused op sequence vs MegaKernel on short ctx (Dao *et al.*, 2022). Integration tests run full L-layer graphs on bucket max ctx (Kwon *et al.*, 2023; NVIDIA, 2024). Fleet canaries track launches/token regressions hourly (Vellaisamy *et al.*, 2026; Chen, 2026). XDNA adds compile-legality tests per bucket (AMD, 2024a,b).

Symbol-level map (GPU). Typical emitted symbols: `megakernel_decode_setup`, `megakernel_decode_step`, `tma_desc_qkv`, `tma_desc_mlp`, `paged_kv_load_block` (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Reviewers grep for `cudaMalloc` inside `decode_step`—there should be none (Chen, 2026; Kwon *et al.*, 2023). Persistent kernels expose `while (true)` loops with cooperative exit flags (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

Table 1.3: Decode benchmark matrix for MegaKernel claims (required metrics).

Variant	Launches/token	Bytes/token	Host ms	Speedup type
Eager PyTorch	800+	Baseline	High	—
CUDA Graphs only	800+	≈Baseline	Low	Orchestration
Partial fusion	100–300	−10–30%	Medium	Mixed
Full MegaKernel	≤1/layer	−30–50%	Low	Residency + orchestr
XDNA static graph	DMA-count	DDR bytes	Low host	Static schedule

Symbol-level map (XDNA). Emitted artifacts include static buffer tables (`aie_buffers.json`), DMA edge lists, and runner entrypoints (AMD, 2024a,b; Chen and YiRage Contributors, 2026). Host code submits `execute` with token index; device-side control flow is data-driven (Chen, 2020; Davies *et al.*, 2025). Illegal overlap errors reference buffer ids—map ids back to IR temporaries in review (Chen and YiRage Contributors, 2026; AMD, 2024a).

1.4 Benchmark Results and Interpretation

Benchmarks that lie. Publishing only "MegaKernel vs. PyTorch eager" without separating **graph capture** from **fusion** repeats the Memory-Floor lesson: 1.259× on Qwen-2.5-7B largely removes host overhead—not HBM bytes (Chen, 2026). Credible **benchmark** tables report launches/token, bytes/token (roofline or profiler), and p99 latency together (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Table 1.3 is the minimum schema for Chapter 11-style **speedup** claims. TaxBreak supplies launches/token anchors (847.5 dense Llama-3.2-1B; 9,305 OL-MoE) (Vellaisamy *et al.*, 2026). Memory-Floor supplies orchestration-only speedup ceilings (Chen, 2026). True MegaKernel **speedup** must show bytes/token movement on the same row—otherwise the result is a graph-capture note, not a fusion chapter (Davies *et al.*, 2025; Dao *et al.*, 2022).

Measurement protocol.

1. Fix BS=1, report context buckets (512, 2048, 8192) separately—KV growth changes residency (Dao *et al.*, 2023; Kwon *et al.*, 2023).

2. Warm graph capture before timing; exclude first-token compile/capture (NVIDIA, 2024; Chen, 2026).

3. Use Nsight Compute (GPU) or DDR byte counters (XDNA)—not wall-clock alone (SemiAnalysis, 2024; AMD, 2024b).

4. Triplet-test GPU, CPU, and NPU backends from one IR export when claiming compiler success (Chen and YiRage Contributors, 2026; Chen, 2020).

Nsight recipe (GPU). Capture one decode token; mark regions for QKV, KV loop, MLP (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). Export `dram_bytes` per region; sum should drop rung-by-rung on the ablation ladder (Chen, 2026; Dao *et al.*, 2022). If only host time drops, you captured a graph—not a MegaKernel (NVIDIA, 2024; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020).

Representative outcomes (literature-aligned). Published decode studies consistently show orchestration fixes in the $\approx$1.2–1.3$\times$ range when HBM traffic is unchanged (Chen, 2026; NVIDIA, 2024). FlashAttention-class fusion reports larger bytes/token reductions on long contexts because attention traffic dominates (Dao *et al.*, 2022, 2023). Full-layer MegaKernels compound both effects when legal on target SRAM—but MoE and wide MLP models hit residency walls first; expect sub-linear **speedup** versus dense Llama baselines (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Multi-hardware reporting. GPU tables alone mislead fleet decisions. Ryzen AI XDNA results should cite DDR bytes/token and compile bucket IDs (AMD, 2024b,a). CPU results should cite L3 miss rates and NUMA-local KV placement (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). A unified compiler narrative (Part VI) only holds when the same IR lowers to all three with explicit legality failures documented—not hidden fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020).

Example 1.4.1. Honest headline: "Full MegaKernel: 2.1$\times$ p50 / 1.4$\times$ p99 vs. eager; 1.25$\times$ explained by graph capture alone (Memory-Floor regime); remaining gain from -42% bytes/token on MLP boundary elimination" (Chen, 2026; Vellaisamy *et al.*, 2026).

Review gate. Reject benchmark PRs that report speedup without launches/token and bytes/token decomposition (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025).

Ablation template. Credible papers and internal reports present a ladder: (A) eager baseline; (B) CUDA Graphs only; (C) FlashAttention + unfused MLP; (D) partial fusion; (E) full MegaKernel (Chen, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Each rung must hold model, checkpoint, BS, context bucket, and clock constant (Davies *et al.*, 2025; NVIDIA, 2024). Skipping rungs hides whether **speedup** came from orchestration or residency (Chen, 2026).

Regression harness. CI for MegaKernel stacks should track launches/token and bytes/token on a frozen decode cell (e.g., Llama-3.2-1B, BS=1, ctx=2048) (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Latency alone drifts with driver versions; byte counters catch silent fusion regressions when someone adds an intermediate `store` (Chen, 2026, 2020). XDNA CI should mirror with DDR byte estimates per bucket (AMD, 2024b,a).

Reporting p99 vs mean. MegaKernels reduce mean ms/token but can worsen tail latency if fusion increases shared-memory bank conflicts or removes overlap opportunities (Davies *et al.*, 2025; Chen, 2026). Always pair mean **benchmark** numbers with p99 on identical traces (Kwon *et al.*, 2023; Dao *et al.*, 2023). Graph capture especially can add first-token spikes—report warm steady-state separately (NVIDIA, 2024; Chen, 2026).

Cross-hardware benchmark row. A complete Table 1.3 row set includes GPU (H100), CPU (AVX-512 server), and XDNA (Ryzen AI) on the same model export (AMD, 2024b; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). GPU may show 2× while XDNA shows compile failure—that failure is a **benchmark** result, not a footnote (AMD, 2024a; Chen, 2020). Industrial compiler chapters must publish legibility matrices, not only peak GPU **speedup** (Davies *et al.*, 2025; Liu *et al.*, 2024).

Energy and watts/token. Edge deployments care about NPU watts/token as much as cloud ms/token (AMD, 2024a,b). MegaKernels that remove DDR trips often dominate energy even when GPU ms/token gains look modest (Chen, 2026; Davies *et al.*, 2025). Include power draw when claiming mobile **speedup** (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

Comparison to vendor baselines. TensorRT-LLM, vLLM, and PyTorch compile paths are moving targets (Kwon *et al.*, 2023; Davies *et al.*, 2025). Freeze

vendor version hashes in **benchmark** appendices (Vellaisamy *et al.*, 2026; Chen, 2026). Custom MegaKernels should beat *vendor fused decode* on bytes/token, not only beat eager Hugging Face (Dao *et al.*, 2022; NVIDIA, 2024).

Worked benchmark narrative. Consider Llama-3.2-1B decode at BS=1, ctx=2048: eager issues ≈848 launches/token (Vellaisamy *et al.*, 2026). CUDA Graphs-only removes ≈3 ms host overhead per Memory-Floor regime (≈1.26×) but leaves bytes/token flat (Chen, 2026). Adding FlashAttention-class attention fusion cuts KV reread traffic (Dao *et al.*, 2022). Full QKV–attn–MLP MegaKernel targets one launch per layer and removes four to six activation HBM round trips—combined **speedup** is multiplicative on orchestration and bandwidth axes only when both move (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). Publish all three numbers; readers will otherwise attribute fusion wins to graphs alone (NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Statistical rigor. Run ≥1000 tokens after warm-up; report confidence intervals on ms/token (Davies *et al.*, 2025; Chen, 2026). Single-trace demos are insufficient for production claims (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). Compare medians and p99 separately—MegaKernels can tighten median while fattening tail if barriers multiply (Kwon *et al.*, 2023; SemiAnalysis, 2024).

Documentation deliverables. Every MegaKernel merge should attach: residency diagram, ablation table, graph capture hash, and IR diff (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026). Missing artifacts revert to "interesting kernel hack," not fleet-ready **implement**ation (Davies *et al.*, 2025; Liu *et al.*, 2024).

Operational playbook (pre-merge).

1. Confirm decode cell: model id, BS, context bucket, precision, SKU stepping (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
2. Capture eager baseline: launches/token, Nsight bytes, p50/p99 ms/token (Chen, 2026; Dao *et al.*, 2022).
3. Capture graph-only baseline on identical cell—quantify orchestration ceiling (NVIDIA, 2024; Chen, 2026).
4. Run fusion ablation ladder (Table 1.3)—no skipped rungs (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

5. Verify numerics vs reference within published tolerance (Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

6. Attach shared-memory/tile SRAM accounting signed by kernel and compiler owners (SemiAnalysis, 2024; AMD, 2024a).

7. XDNA/CPU: repeat bytes/token axes with target-appropriate counters (AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020).

8. File graph capture hash and driver version in manifest (NVIDIA, 2024; Kwon *et al.*, 2023).

9. Load-test p99 under concurrent sessions—MegaKernels can contend on L2/shared (Davies *et al.*, 2025; Chen, 2026).

10. Canary in fleet with auto-rollback on metric regression (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Anti-patterns in MegaKernel benchmarks. Quoting prefill throughput for decode chapters (Dao *et al.*, 2023; Davies *et al.*, 2025). Comparing against outdated eager without FlashAttention (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Reporting GPU-only **speedup** while XDNA compile fails silently in production (AMD, 2024a; Chen and YiRage Contributors, 2026). Tuning MMA tiles without measuring bytes/token movement (Chen, 2026; SemiAnalysis, 2024). Publishing mean ms/token without p99 under concurrent load (Kwon *et al.*, 2023; Chen, 2026). Omitting MoE routing overhead in OLMoE comparisons (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Using tiny ctx in demos while selling long-context products (Dao *et al.*, 2023; Kwon *et al.*, 2023). Claiming MegaKernel fusion when only CUDA Graphs changed (Chen, 2026; NVIDIA, 2024). These anti-patterns recur in vendor blogs—this chapter's tables exist to reject them in internal review (Davies *et al.*, 2025; Chen, 2020).

Closing metric reminder. Before closing any MegaKernel milestone, paste three numbers into the PR description: launches/token, bytes/token, p99 ms/token at BS=1 on the declared context bucket (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). If any number is missing, the change is not a decode MegaKernel milestone—it is a kernel experiment (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff to Chapter 12. Execution modes (static capture, persistent kernels, JIT specials) decide *when* to pay compile costs vs runtime flexibility (NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b). Chapter 11 fixed *what* to fuse inside a decode layer; Chapter 12 maps product constraints to mode selection (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Keep bytes/token and launches/token columns attached when debating modes—mode debates without metrics reinvent Chapter 1 failures (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022).

Maintenance. Model weight updates (LoRA merge, quantization recalibration) invalidate captures and static XDNA graphs (Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2024). Version MegaKernel artifacts with checkpoint hashes and bucket ids (Davies *et al.*, 2025; Kwon *et al.*, 2023). Fleet rollouts without artifact versioning cause silent quality regressions unrelated to fusion math (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023).

Final acceptance test. Given identical weights and KV state, MegaKernel output must match eager within documented tolerance for 10^4 random tokens (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Performance acceptance requires Table 1.3 rows for at least two context buckets (Dao *et al.*, 2023; Kwon *et al.*, 2023). Ships only when both pass (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b).

Stakeholder map. Kernel engineers own tile shapes and barriers (SemiAnalysis, 2024; NVIDIA, 2022). Compiler engineers own fusion boundaries and buffer lifetimes (Chen and YiRage Contributors, 2026; Chen, 2020). Serving engineers own bucket policies and fleet rollouts (Kwon *et al.*, 2023; Davies *et al.*, 2025). Benchmark engineers own ablation ladders and regression harnesses (Vellaisamy *et al.*, 2026; Chen, 2026). Reviews fail when any stakeholder optimizes their metric in isolation (Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2024).

Release checklist. Every MegaKernel PR must attach: peak shared-memory accounting diagram; ablation table row for graph-only vs full fusion; triplet bytes/token for GPU/CPU/NPU; p99 ms/token at BS=1 for context buckets 512/2048/8192 (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen

and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Missing artifacts revert the change to kernel experiment status (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

1.5 Key Takeaways

- **MegaKernels collapse two bills.** Target O(1) launches per layer *and* eliminate activation HBM round trips—graphs alone pay only the host bill (Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024).
- **Full-layer residency envelope.** QKV, online softmax attention, norms, and MLP share one SRAM/tile budget per decode step (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).
- **Buffer accounting before merge.** Peak simultaneous tensors must fit SM shared memory or XDNA tile SRAM; failed legality is a blocker (AMD, 2024a; Chen, 2020).
- **Benchmark honestly.** Report launches/token, bytes/token, and p99 ms/token; separate orchestration speedups from fusion speedups (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
- **Context buckets are compile parameters.** KV loop trip counts change each token; capture policies must match max context buckets (Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2023).
- **Triplet regression.** One IR, GPU/CPU/NPU cells—never ship GPU-only residency tables to Ryzen AI fleets (AMD, 2024b; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

1.6 Conclusion

Decoder MegaKernels exist because FlashAttention and CUDA Graphs each paid only part of the batch-1 decode bill (Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026). Section 1.1 defined the fused-layer envelope; Section 1.2 bound QKV, online softmax, and MLP under one buffer plan; Section 1.3 specified device-loop invariants; Section 1.4 demanded metric tables that expose which bill was actually paid (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Treat every op boundary as a billable HBM round trip at batch-1 (Chen, 2026; Goodfellow *et al.*, 2016). Chapter 12 compares execution modes—static capture, persistent kernels, JIT specials—that decide *when* these fused shapes run in production (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024). This chapter's narrative mirrors OŔeilly-style performance engineering: name the constraint (launch storms *and* HBM round trips), quantify with TaxBreak and Memory-Floor, co-design software with Hopper TMA and XDNA tile rules, then gate releases on profile-driven metrics rather than peak FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022). Carry peak buffer accounting, context buckets, and triplet regression forward; Paged KV block tables, rotary embeddings, and sampling logic must either live inside the captured MegaKernel body or be proven off the critical path with measured ms/token impact (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022). MegaKernel work is done when metrics prove fusion on representative batch-1 cells at production context lengths with published ablation tables and Nsight byte counters attached (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a).

PR metric gate. Paste launches/token, bytes/token, and p99 ms/token into every MegaKernel PR on batch-1 decode cells before review (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 3, 4, 5, 6, 7, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 7, 8, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 3, 4, 6, 7, 8, 9, 10, 11, 13, 14, 18, 19, 20, 21, 22, 23, 24, 25

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 8, 10, 11, 12, 13, 14, 17, 18, 19, 20, 21, 22, 23, 24, 25

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 19, 20, 22, 23, 24, 25

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24, 25

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 2, 3, 4, 6, 7, 9, 10, 11, 12, 13, 14, 16, 19, 20, 21, 22, 23, 24, 25

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 17, 23, 25

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23, 24, 25

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 21, 22, 23, 24, 25

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25